

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №2

Тема: Міри складності алгоритмів. Класи задач P і NP

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Навчитися визначати обчислювальну складність алгоритмів у нотації "О-велике", застосовувати правила обчислення складності для типових конструкцій (цикли, вкладені цикли, розгалуження), а також розрізняти класи задач P та NP на прикладах.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal — на вибір викладача), текстовий редактор для оформлення звіту.

Короткі теоретичні відомості

1. Складність алгоритму — це кількісна оцінка ресурсів (часу або пам'яті), необхідних для виконання алгоритму залежно від розміру вхідних даних n .

- **Часова складність** — кількість елементарних операцій.
- **Просторова складність** — обсяг пам'яті, що використовується.

2. Асимптотичні позначення:

Позначення	Назва	Зміст
$O(g(n))$	"О-велике"	Верхня межа зростання (найгірший випадок)
$\Omega(g(n))$	"Омега"	Нижня межа зростання (найкращий випадок)
$\Theta(g(n))$	"Тета"	Точна асимптотична межа

3. Основні класи складності (за зростанням):

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

4. Правила обчислення складності:

- Послідовні оператори — складності додаються: $O(f) + O(g) = O(\max(f, g))$
- Вкладені цикли — складності перемножуються: $O(f) \times O(g)$
- Розгалуження — обирається гілка з більшою складністю
- Константи та доданки нижчого порядку відкидаються: $O(3n^2 + 5n + 2) \rightarrow O(n^2)$

5. Клас P (Polynomial) — множина задач, які можна розв'язати за поліноміальний час $O(n^k)$ на детермінованій машині (наприклад: сортування, пошук, множення матриць).

6. Клас NP (Nondeterministic Polynomial) — множина задач, для яких розв'язок можна перевірити за поліноміальний час, але сам розв'язок може вимагати експоненціального часу пошуку (наприклад: задача комівояжера, задача про рюкзак, 3-SAT).

7. Співвідношення P і NP: Очевидно, що $P \subseteq NP$. Питання, чи $P = NP$, є однією з семи "задач тисячоліття" (Millennium Prize Problems) і залишається відкритим.

8. NP-повні задачі — найскладніші задачі класу NP, до яких за поліноміальний час зводиться будь-яка інша задача цього класу (задача виконуваності булевих формул SAT, задача про клік, розфарбування графа).

Хід роботи

Завдання 1. Визначення асимптотичної складності фрагментів коду

Для кожного фрагмента алгоритму (псевдокод нижче) визначити асимптотичну складність у нотації O-велике, обґрунтувавши підрахунком кількості операцій.

a)	б)	в)
for i = 1 to n	for i = 1 to n	i = n
S = S + i	for j = 1 to n	while i > 1
	S = S + i*j	i = i / 2
		k = k + 1
г)	г)	
for i = 1 to n	for i = 1 to n	
for j = 1 to i	for j = 1 to n	
S = S + 1	for k = 1 to n	
	S = S + 1	

Порядок виконання:

1. Візьміть фрагмент (а). Визначте, скільки разів виконується тіло циклу — воно виконується рівно n разів, тому що i пробігає значення від 1 до n без вкладеності. Запишіть висновок: $O(n)$.

2. Перейдіть до фрагмента (б). Спочатку порахуйте, скільки разів виконується внутрішній оператор $S = S + i*j$ для одного фіксованого i (це n разів, оскільки j також пробігає від 1 до n). Потім помножте це на кількість значень i (теж n). Отримаєте $n \cdot n = n^2$ виконань. Запишіть: $O(n^2)$.

3. Перейдіть до фрагмента (в). Зверніть увагу, що цикл `while` не залежить від n лінійно – на кожній ітерації i ділиться навпіл. Порахуйте, скільки разів число n можна поділити навпіл, перш ніж воно стане ≤ 1 (це i є визначення логарифма за основою 2). Запишіть: $O(\log n)$.

4. Перейдіть до фрагмента (г). Тут внутрішній цикл залежить від зовнішнього (j пробігає від 1 до i , а не до n). Обчисліть суму $1 + 2 + 3 + \dots + n$ за формулою арифметичної прогресії $n(n+1)/2$, спростіть до $O(n^2)$ (оскільки константи і доданки нижчого порядку відкидаються).

5. Перейдіть до фрагмента (г). Тут три вкладені цикли, кожен з яких незалежно пробігає від 1 до n . Перемножте три складності: $n \cdot n \cdot n = n^3$. Запишіть: $O(n^3)$.

6. Оформіть відповіді у вигляді таблиці "Фрагмент — Кількість операцій (розгорнуто) — Спрощена складність $O(\dots)$ " з коротким текстовим поясненням до кожного пункту.

Завдання 2. Порівняльний аналіз

Дано алгоритми з часовою складністю $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$. Розрахувати (в таблиці) орієнтовний час виконання кожного алгоритму для $n = 10$, $n = 100$, $n = 1000$, якщо одна елементарна операція виконується за 10^{-6} с. Зробити висновок про практичну застосовність кожного класу складності при зростанні n .

Порядок виконання:

1. Створіть у зошиті/текстовому редакторі таблицю з рядками $n = 10, 100, 1000$ і стовпцями $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$.
2. Для кожної клітинки підставте відповідне n у формулу складності: наприклад, для $O(n)$ при $n = 100$ отримаєте 100 операцій; для $O(n^2)$ при $n = 100$ отримаєте 10 000 операцій; для $O(2^n)$ при $n = 100$ отримаєте 2^{100} (величезне число — досить показати степінь, обчислювати повністю не потрібно).
3. Помножте кожне отримане число операцій на 10^{-6} с, щоб отримати орієнтовний час у секундах.
4. Переведіть отримані секунди у зручні одиниці (мс, с, хв, роки) — особливо для $O(2^n)$, де час при $n = 100$ виявиться астрономічно великим.
5. Занесіть результати в таблицю.
6. Напишіть висновок (5–7 речень): як швидко зростає час виконання для кожного класу складності зі збільшенням n , чому алгоритми з експоненціальною складністю є непридатними для великих n , і чому різниця між $O(n)$ та $O(n \log n)$ практично малопомітна при малих n , але стає суттєвою при великих.

Завдання 3. Задачі класів P і NP

Заповнити таблицю, віднісши кожну із запропонованих задач до класу P або NP (із зазначенням, чи є вона NP-повною), коротко обґрунтувавши відповідь:

1. Сортування масиву чисел
2. Задача комівояжера (Traveling Salesman Problem)
3. Пошук найкоротшого шляху в графі (алгоритм Дейкстри)
4. Задача про рюкзак (Knapsack Problem)
5. Перевірка, чи є число простим
6. Розфарбування графа у k кольорів
7. Пошук елемента в відсортованому масиві

Порядок виконання:

1. Створіть таблицю зі стовпцями: "№", "Задача", "Клас (P / NP)", "NP-повна? (так/ні)", "Обґрунтування".
2. Для кожної задачі поставте собі запитання: "Чи існує відомий алгоритм, який розв'язує цю задачу за поліноміальний час?" Якщо так — задача належить до P (наприклад, задачі 1, 3, 5, 7 мають відомі поліноміальні алгоритми: сортування злиттям/швидке сортування, алгоритм Дейкстри, тест простоти АКС/пробне ділення, бінарний пошук).
3. Якщо поліноміального алгоритму розв'язання невідомо, але **перевірити** запропонований розв'язок можна швидко (за поліноміальний час) — задача належить до NP (задачі 2, 4, 6).

4. Визначте, які з NP-задач є NP-повними (задача комівояжера та розфарбування графа — класичні приклади NP-повних задач; задача про рюкзак у загальній постановці також NP-повна).

5. У колонці "Обґрунтування" одним реченням поясніть свій вибір (наприклад: "Сортування — відомий алгоритм quicksort/mergesort працює за $O(n \log n)$, отже P").

6. Звірте результат із теоретичним матеріалом роботи та за потреби скоригуйте.

Завдання 4. Практичне вимірювання

У обраному середовищі програмування реалізувати два алгоритми пошуку максимального елемента масиву:

- лінійний перебір ($O(n)$);
- турнірний метод з рекурсивним поділом масиву навпіл ($O(n)$ з іншою константою).

Виміряти реальний час виконання для масивів розміром 10^3 , 10^4 , 10^5 , 10^6 елементів (функціями таймера мови програмування) та побудувати графік залежності часу від n .

Порядок виконання:

1. Створіть новий проєкт/файл у обраному середовищі програмування.
2. Напишіть функцію `linear_max(arr)`, яка одним проходом циклу `for` порівнює кожен елемент з поточним максимумом і повертає найбільше значення.
3. Напишіть рекурсивну функцію `tournament_max(arr, left, right)`: якщо в піддіапазоні один елемент — поверніть його; інакше поділіть діапазон навпіл, рекурсивно знайдіть максимум у лівій і правій половині і поверніть більший з двох.
4. Напишіть допоміжну функцію генерації масиву випадкових чисел заданого розміру n (використайте `random/rand`).
5. Підключіть модуль вимірювання часу (наприклад, `time.perf_counter()` у Python, `chrono` у C++, `GetTickCount/DateTime` у Pascal).
6. Для кожного розміру $n \in \{1000, 10000, 100000, 1000000\}$: згенеруйте масив, зафіксуйте час до і після виклику `linear_max`, обчисліть різницю; повторіть те саме для `tournament_max`.
7. Занесіть отримані пари $(n, \text{час_linear}, \text{час_tournament})$ у таблицю.
8. Побудуйте графік (в Excel, Python/matplotlib або вбудованих засобах середовища) з віссю X — розмір масиву n , віссю Y — час виконання в мілісекундах, двома лініями для обох алгоритмів.
9. Порівняйте отримані криві з теоретичною лінійною залежністю $O(n)$ і поясніть у висновку, чому обидва алгоритми мають однаковий порядок зростання, але різняться за абсолютним часом (константа прихована в O -нотації).

Контрольні питання

1. Що таке обчислювальна складність алгоритму?
2. У чому різниця між часовою та просторовою складністю?
3. Поясніть різницю між нотаціями O , Ω та Θ .
4. Як обчислюється складність вкладених циклів?

5. Дайте визначення класу P.
6. Дайте визначення класу NP. Чим воно відрізняється від визначення класу P?
7. Що означає термін «NP-повна задача»?
8. Наведіть приклади задач класу P та класу NP.
9. У чому суть проблеми "P проти NP"?

Зміст звіту

1. Тема, мета роботи.
2. Розв'язання завдань 1–3 з розгорнутими обчисленнями та обґрунтуванням.
3. Код програми, таблиця виміряних результатів і графік до завдання 4.
4. Висновки.